# Take-Home Assignment

## Senior Data Scientist Role @Mobupps

Recommendation Modeling and Experimentation

MobUpps | AI Department

---

| | |
|---|---|
| **Time expected** | ~5 hours of focused work |
| **Deadline** | 7 calendar days from receipt |
| **Submission** | Git repository |
| **Language** | Python; all code, comments, commits and README in English |
| **What we evaluate** | Modeling judgment, evaluation rigor, and clarity of reasoning, not model accuracy |

## 1. Context

MobUpps is a global Ad-Tech company powering data-driven user acquisition and engagement at scale. Our AI Department builds production machine learning across the business: recommendation and personalization systems, GenAI products such as an internal analytics assistant that answers questions over real data, internal data platforms, and DMP capabilities.

Beyond this current work, we are building our next product: a personalized recommendation engine that, given a user, serves a ranked list of apps they are likely to engage with next, in production and at real scale. The Applied Data Scientist will take a central part in building it. For that reason, this exercise focuses on the recommendation modeling and experimentation at the heart of that product.

The exercise is small on purpose. We are not testing whether you can build a full production recommender in a week. We are testing how you frame the problem, choose and justify a model, evaluate it honestly, and reason about taking it from offline to online. Please read this entire document before you start.

## 2. Your Task

Given a user_id, return a ranked list of N apps the user is likely to engage with next. Concretely, you will deliver:

1. **A data layer** that loads, validates, and prepares the provided dataset for modeling.
2. **One recommendation model.** One is enough. A simple, well-justified model is strongly preferred over a complex one with thin reasoning. Spend your modeling time on choosing and justifying, not on tuning.
3. **An evaluation** you actually ran, and that you can defend.
4. **A written analysis** (Section 5). This is the single most important deliverable.

Your model must expose a class with `fit(train_df)` and `recommend(user_id: str, n: int) -> list[str]` so it runs against the provided `scripts/evaluate.py`. Unknown or cold users must be handled gracefully: your call on what to return, but it must not crash.

**On scope.** While we value and expect solid working knowledge of production engineering, services, containerization, and deployment, we have deliberately left these outside the scope of this exercise. We did this both to respect everyone's time and to focus the assignment squarely on the level of excellence this role demands. For this task there is therefore no HTTP service, no Docker, no API, and no frontend: a runnable model together with your written analysis is the entire deliverable.

## 3. What We Provide

The assignment package contains:

- `data/users.csv`: ~10,000 users with profile attributes (signup_date, country, device, age_bucket).
- `data/apps.csv`: ~500 apps with categorical and numeric features (category, size_mb, age_rating, avg_rating, total_ratings).
- `data/interactions.csv`: ~490,000 user-app interactions over ~90 days, with timestamps, event type (view, click, install, open, plus post-install events) and duration.
- `scripts/evaluate.py`: a ready-to-use evaluation harness that performs a time-based train/test split and computes Recall@10 and NDCG@10, with a popularity baseline included as a reference.

We expect you to load, validate, and prepare these tables for modeling, and to understand exactly what the evaluation script measures. You may use evaluate.py as-is, modify it,

replace it, or ignore it. Using it is optional. Understanding what it measures, and what it does not, is not.

**Reminder: we are not evaluating you on offline metrics.** A Recall of 0.05 with sharp reasoning beats a Recall of 0.30 from a black box with a thin write-up.

## 4. Code Deliverables

- **Data loading and preparation** that reads from a data/ directory (assume it is mounted at the path you document).
- **A recommendation model:** fitting logic plus inference logic. Same module or separate is your call.
- **An evaluation** script or notebook showing the numbers you report and how you produced them.
- **Python and SQL:** your modeling and data preparation should be in clean, production-grade Python. In addition, although the data ships as CSV files, assume that in production these tables live in a SQL warehouse. Include one or two SQL queries you would use to build a key feature or training table (for example, per-user interaction aggregates), so we can see both your Python and your SQL.
- **A README** with exact, copy-pasteable commands to set up and run everything, assuming a reviewer with no prior context.

Notebooks are fine for exploration, but the model and evaluation logic should be runnable as scripts, not buried in a notebook.

## 5. Written Analysis: the core of this exercise

Answer the five questions below, in this order, with these headings. Keep it concise: roughly 1 to 2 pages total. We value tight reasoning over length, and we will probe these answers in the interview, so write only what you can defend.

### 5.1 Target and signal definition

The interaction log contains several event types of different strength. Define precisely what your model treats as a positive signal, and justify it. Do weaker events (for example a view) count? You never observe explicit negatives: how do you treat unobserved user-app pairs, and what does that assumption cost you?

### 5.2 Model choice and the one you rejected

Justify your choice of model family for THIS dataset specifically. Reason from the shape and density of the data, not from the general popularity of methods. Then name one model family you deliberately did not use, and explain why it would underperform here. “It is state of the art” is not a reason.

### 5.3 Critique the evaluation

Using Recall@10 and NDCG@10 on the provided split: what does this evaluation fail to capture about whether your model is actually good for MobUpps’ business? Identify at least one risk in the split itself. Would you trust these numbers to decide a production rollout, and why or why not?

### 5.4 Hyperparameters: predict, don’t list

For the model you built, pick the two hyperparameters that matter most. For each: what does it control; in which direction does the offline metric move as you increase it; and, separately, what happens to real recommendation quality? Describe the failure mode at each extreme. Be specific to your model. We will probe this live.

### 5.5 From offline to online

Your model beats the popularity baseline offline. Design the online A/B test that decides whether it ships: the hypothesis, the primary metric and why that one, the unit of randomization, how you would size the experiment, two guardrail metrics, and the single biggest threat to the validity of the result.

## 6. Data-Quality Findings

Document the data-quality issues you find: **what** you found, **how** you found it, and whether you **fixed, flagged, or ignored** each one, with a one-line justification. You do not have to fix everything. We are looking for the judgment in what you chose to address and why.

## 7. Constraints and Ground Rules

- **Time budget: ~5 hours of focused work.** If you reach hour 8, stop and submit. We treat large overruns as a signal, not a strength.
- **No service, no Docker, no frontend.** A runnable model plus your written analysis is the entire deliverable.
- **Do not micro-tune.** Pick something reasonable, justify it, move on. We are not scoring your offline metric.

- **AI assistants:** assume we assume you used them. We do not penalize this. We do expect you to explain every line, defend every decision, and modify your code live in the follow-up interview. The interview will catch shallow understanding.
- **Allowed libraries:** any open-source Python library. No paid APIs.

## 8. Submission

- Submit a private Git repository (GitHub or GitLab), share access with shlomit@mobupps.com.
- **Do not** commit the dataset itself. Assume the reviewer mounts the `data/` directory at the path you document.
- Repository structure is your call: we want to see how you organize a project. Include the data layer, the model (fit and recommend), the evaluation you ran, and a README. The written analysis can live in the README or in a separate ANALYSIS.md.

## 9. What Happens After You Submit

Within five business days you will receive either an invitation to a ~60-minute technical interview with the Head of AI and the CTO, centered entirely on your submission, or a decline with brief feedback. Either way, we will respond. We respect your time.

## 10. Final Note

This exercise is calibrated to reveal how you actually reason, not how you present yourself on a CV. The candidates who do best are not the ones who build the most impressive model in the time available. They are the ones who scope tightly, evaluate honestly, and reason clearly about what they did and did not do.

Good luck. We look forward to reading your submission.